Sistemas operativos

Jaime Nazar Anchorena

Contents

Practica 1: POSIX **5**
- Entorno de desarrollo . . . 5
- Crear procesos . . . 5
- `ps` . . . 6
- Copy on write . . . 6
- File management . . . 6
- `pipe` . . . 8
  - dup . . . 8

Practica 2: Herramientas de trabajo **8**
- Unit testing . . . 8
  - Unit test . . . 8
  - Test driven development . . . 9
  - Testing en C . . . 9
  - CuTest . . . 9
  - Unit test structures & patterns . . . 9
  - Git hooks . . . 9
- Workflow GIT . . . 9
  - Branch . . . 10

Practica 3: TP02 **10**
- Memory management . . . 11
  - Responsabilidades del memory manager . . . 11
  - Consideraciones para el TP . . . 11
  - Buddy allocation . . . 11

Practica 4: Context switch, scheduling y procesos **11**
- Crear un proceso . . . 11
  - Stack . . . 11
  - RIP . . . 12
  - Si no hay proceso ready? . . . 12

Practica 5: Sincronizacion **12**
- Dentro de la region critica . . . 12
- Soluciones . . . 12

Spin lock . . . 12

Teorica 1: Introduccion **13**

Sistemas operativos . . . 13
Administrador de recursos . . . 13
Revision hardware . . . 13
Procesador . . . 13
Memoria . . . 13
Dispositivos I/O . . . 14
Booteo . . . 15
Tipos de SO . . . 15
System calls . . . 16
Read . . . 16
Waitpid . . . 16
Kill . . . 16
Fork . . . 17
A donde va la salida estandar . . . 17
Estructura de un sistema operativo . . . 17
Monolithic systems . . . 17
Microkernels . . . 18
Virtual machines . . . 18

Teorica 2: Procesos **19**

Pseudo concurrencia . . . 20
Reflexion . . . 20
Creacion de procesos . . . 21
Win32 . . . 21
Terminacion de procesos . . . 22
Jerarquia de procesos . . . 22
Grupo de procesos . . . 22
Win32 . . . 22
Estado de procesos . . . 22
Transcisiones . . . 23
Implementacion de procesos . . . 23
Modelado multiprogramacion . . . 24

Teorica 3: Inter process communication(IPC) **24**

Memoria compartida . . . 25
Pasaje de mensajes . . . 26
Sincronizacion . . . 26
Buffering . . . 26
Ejemplos de IPC . . . 26
Files . . . 26
Pipes . . . 26
Files vs. pipes . . . 27
Sockets . . . 27

FIFO vs Unix Domain Sockets . . . 27
Signals . . . 27
Race condition . . . 28
Region critica . . . 28
Mecanismos de sincronizacion . . . 28
Deadlock . . . 28

Teorica 4: Threads **28**
Modelo de threads . . . 29
Proceso vs. thread . . . 29
Implementacion en espacio de usuario . . . 30
Ventajas . . . 30
Desventajas . . . 30
Implementacion en espacio de kernel . . . 30
Implementacion hibrida . . . 30
Schedulers activators . . . 31

Teoria 5: Scheduling **31**
Clasificacion . . . 31
Non-preemptive . . . 31
Preemptive . . . 31
Categorias . . . 32
Objetivos . . . 32
Algoritmos . . . 33
Batch: First-Come First-Serve . . . 33
Batch: Shortest job first . . . 33
Batch: Shortest remaining time next . . . 33
Interactivo: Round-Robin . . . 33
Interactivo: Priority scheduling . . . 34
Interactivo: Priority scheduling - Multiple queues . . . 34
Interactivo: Priority scheduling - Shortest process nect . . . 34
Interactivo: Guaranteed scheduling . . . 35
Interactivo: Lottery Scheduling . . . 35
Interactivo: Fair-share scheduling . . . 35

Teorica 6: Memoria **35**
Problemas que pueden ocurrir . . . 36
Cada proceso accede directo a memoria fisica . . . 36
Espacio de direcciones . . . 37
Como se implementa memoria virtual . . . 37
Como se resuelven los problemas . . . 37
Paginacion . . . 38
Tabla de paginas . . . 38
Page fault . . . 39
No todo es color de rosa . . . 39
Analisis de costos . . . 40

TLB 40
Tabla de paginas multinivel 40
Tabla de paginas invertidas 41
Algoritmo de reemplazo de paginas 41
Optimo 41
Not recently used(NRU) 41
First-in first-out(FIFO) 42
Second Change(SC) 42
Clock(C) 42
Least recently used(LRU) 42
Not frequently used(NFU) 42

Practica 1: POSIX

Entorno de desarrollo

Trucos de bash:

- `ctrl + r` para buscar historial
- `!!` expande con el ultimo comando (`$sudo !!`)
- `file.{jpg, png}` expande a `file.jpg file.png`
- `$(...)` da el output de un comando (`touch file-$(date -I)`)
- `ctrl + a` para ir al comienzo de la linea
- `ctrl + e` para ir al final de la linea
- `cd -` vuelve al dir que estabas antes
- `fc` fix command, abre el ultimo comando en un editor y corre la version editada
- `type` indica de si algo es builtin, program, o alias (`type ping`)

En bin/ estan todos los comandos de administracion, para configurar el sistema. Por otro lado, en bin/ estan los comandos de uso general.

Crear procesos

Se pueden crear utilizando `fork`, syscall que la llamo una vez y vuelve dos veces, el proceso original y el que se creo.

Los dos procesos son identicos salvo el valor de retorno de `fork`. Si `fork` no puede crear un proceso(raro, pero puede pasar), devuelve -1.

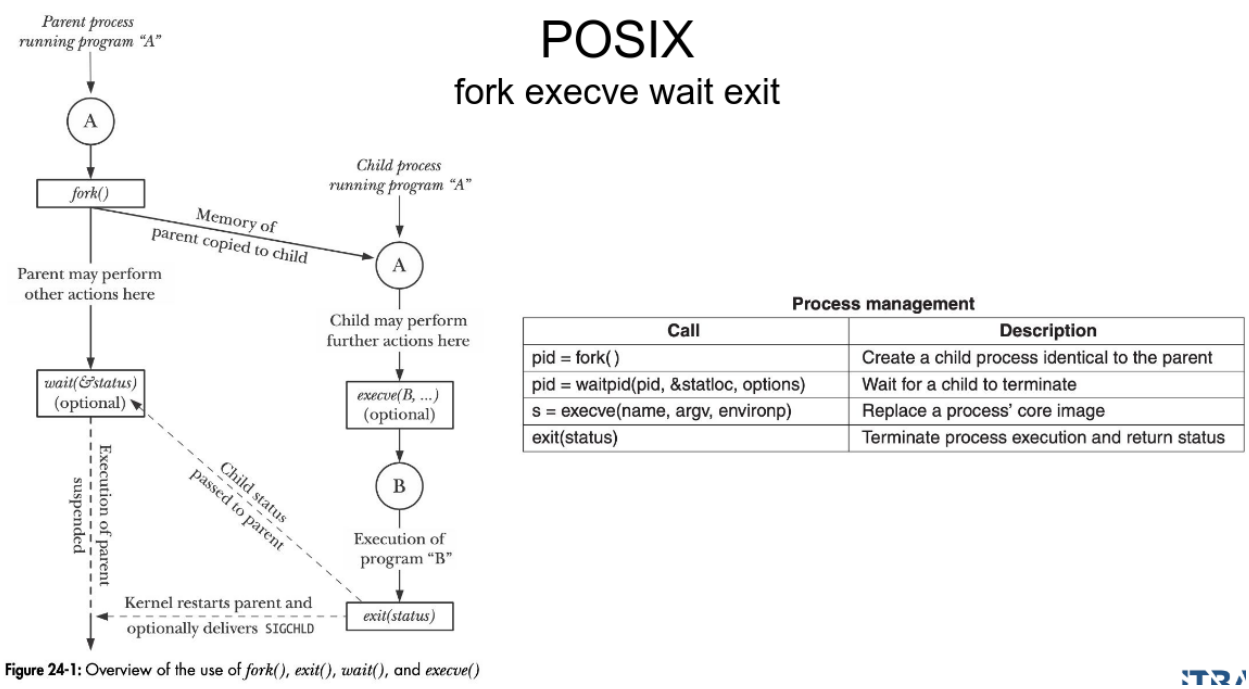

Figure 1: Pipeline fork

Como cada proceso puede crear varios procesos, pero tiene un solo padre, si quiero saber cual es el `pid` del

padre es necesario que el padre sepa cual es el `pid` del hijo. Esto se logra devolviendo el `pid` del hijo en el padre.

Es muy importante la llamada a `wait` porque sino queda un monton de procesos **undefined**, consume entradas en la tabla de procesos. Todas las variables que tenga un proceso original, incluyendo los punteros, el hijo los hereda. Es importante por ese motivo cerrar todos los **file descriptor**.

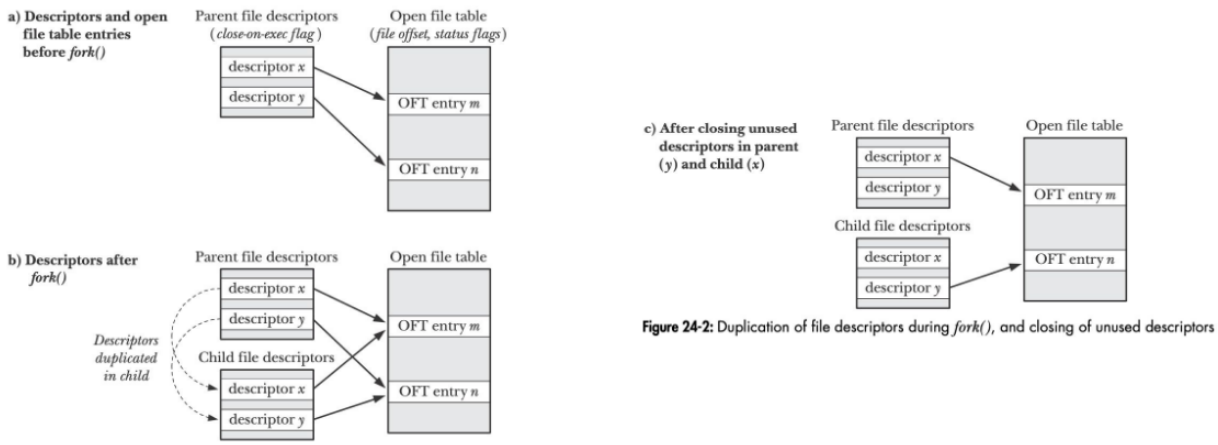

Figure 2: File descriptors fork

`ps`

Muy recomendado que lo aprendamos a usar bien para debuggear los TP. Se corre generalmente con `aux` pued indica: a+x -> mostrar todos los procesos, u -> incluir la columna con informacion del usuario de cada programa. Se puede combinar con otros arg(todo junto):

- Muestra los procesos que estan corriendo en el momento. Usando `w` como argumento(wide) muestra los argumentos que se utilizaron para correr cada programa.

- Si se corre con `e` va a mostrar las variables de entorno.
- Si se corre con `f` se mostrara un arbol de los procesos con caracteres ASCII

  Nota: Hay mas arg en la presentacion de campus.

Copy on write

Los procesos clones usan la misma memoria que el padre(heap, stack, memory maps), copiar esa memoria cada vez que usamos fork es lento y ocupa memoria. Entonces, linux permite que ambos procesos utilicen la misma memoria fisica y solo copia la memoria cuando alguno de los procesos intenta escribir. Linux implementa esta dandole la misma **page table** a cada programa, pero marcadas como **read only**. Entonces, cuando alguno intenta escribir ocurre una **page fault**.

File management

Todo lo que es I/O es universal en Unix, se maneja como si fuera un archivo. Es muy eficiente el read write, lo unico que cambia entre tipos como lo accedo. Para trabajar con archivos como humanos les asignamos

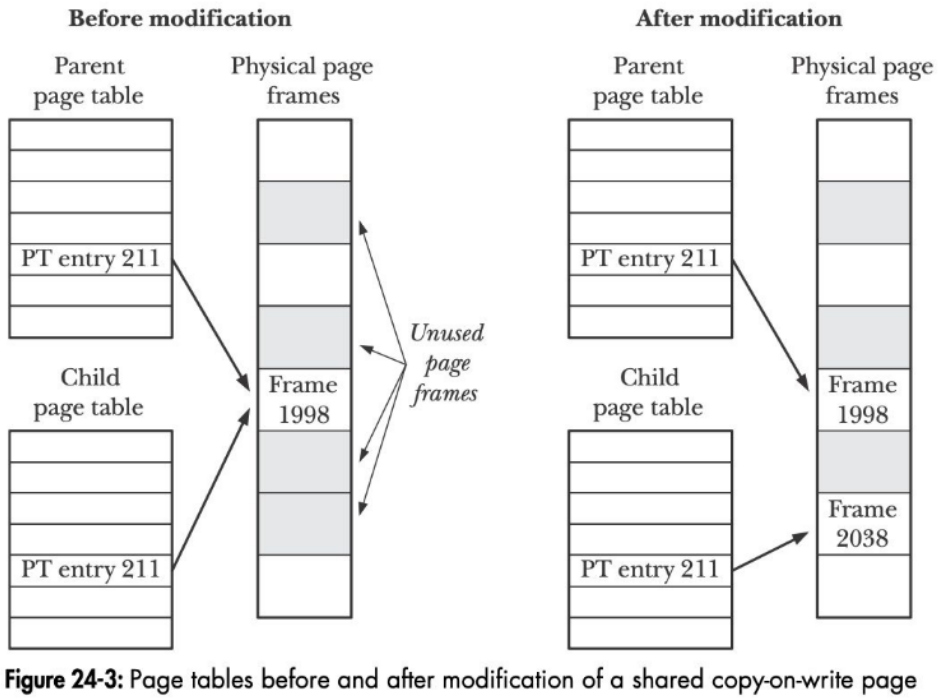

Figure 3: Representacion de copy on write

nombres, pero las computadoras prefieren tratar con numeros. Por lo tanto, cuando trabajamos con bajo nivel utilizamos **file descriptors** los cuales indican la referencia a un archivo **abierto**.

Una vez con el fd se puede leer y escribir, el fd 0 es la salida estandar. Comandos utiles:

- `head`, primeras lineas de un archivo
- `tail`, ultimas lineas de un archivo
- `stat`, muestra metadata del archivo
- `lseek`, corre el puntero a otra posicion

Una vez termino de trabajar con un fd, se libera y el numero queda disponible para cuando abra algun otro archivo. Cuando abro un archivo, usa el fd mas chico disponible. Cuando se lee de disco es muy importante que lo que uno lea o escriba este alineado con el **tamano del bloque** para evitar fragmentacion, no es obligatorio, pero es una buena practica.

Un archivo tiene un formato predefinido? No, para el sistema operativo no importa, hay metadata guardada en disco para saber su ubicacion o permisos, pero el archivo en si son bytes para el SO. Es problema de la aplicacion saber que formato tiene, si tubieran formatos para el SO seria demasiado complejo e innecesario.

> **Nota**: Cada proceso tiene su tabla con el mapeo de fd a nombre, es un error muy comun tener una tabla global para todos los procesos(esta mal eso). Todo lo que comentan es **local** al proceso. Si dos procesos tienen abierto el mismo archivo el fd no tienen porque coicidir.

`pipe`

Es un canal donde se envia todo lo que saca un proceso se lo manda a otro, es unidireccional y es un metodo de comunicacion. Es el mas sencillo que hay. Es un buffer compartido, donde un proceso escribe y otro lee, una cola. No hay posibilidad de reabrir un pipe, pues es un objeto anonimo(no tiene nombre).

En caso de que el programa que lee termine y el otro siga escribiendo se lanzara una senal de error nivel sistema operativo. No es comportamiento normal, por lo tanto se lanza un error. Detecta que el otro programa dejo de leer pues desaparece el fd. Pero que pasa si deja de escribir el programa, que pasara con el que lee? Se vacia el pipe y cuando en ultimo read devuelva 0, entonces se cierra el pipe.

Otro caso, el proceso que lee, lee despacio, entonces, se llena el buffer. Cuando la otra parte quiera escribir y el SO ve que el buffer esta lleno, bloquea la parte que escribe hasta que se pueda ejecutar write. No da error, pues esto es una situacion normal.

dup

El manejo del pipe se realiza de la siguiente manera:

1. Corro pipe en shell. Se llama syscall `pipe`
2. `pipe` devuelve a la shell fd escritura y fd lectura
3. Se hace un fork, de manera que los hijos(las dos shell) tienen los mismo fd que el padre(pues se hacen copias identicas).
4. La shell cierra los dos fd, pero siguen abiertos para los hijos. Se separa del pipe.
5. Se elimina el fd de lectura de un hijo y el de escritura para el otro. Quedando el diagrama de pipe convencional.
6. Luego para el proceso 1 se configura el stdout como el pipe y para el proceso 2 es el stdin. Duplicando los fd, cerrando stdout y stdin para los proceso.
7. Una vez duplicados a std y stdin, se cierran los originales.
8. Se hace el exec en las shell.

Aca child 1 y child 2 son una shell, al final del todo se hace el exec y se ejecutan los programas. Esta es una de las razones por la cual stderr esta separado de stdout.

> **Nota**: Para ver la representacion grafica ver apuntes. Puede haber dos fd, uno duplicado, abiertos, pero es mala practica.

Practica 2: Herramientas de trabajo

Unit testing

No nos lo van a pedir, pero sirve un monton. Puede haber un bug en el codigo, pero tambien puede haber un bug en el codigo que prueba el codigo. Para evitar esta situacion hay varias tecnicas.

Unit test

Se utilizan para probar por separado todas las partes de forma independiente, si se cambia el codigo entonces se pueden detectar errores.

> Todo junto se escribe separado.

Sin embargo, puede ocurrir que se pasen todos los test y al integrar no funcione nada. De manera que hay que prestar especial atencion a la integracion.

Test driven development

Se utilizo en otros cuatrimestras, pero fue descartado. Basicamente se comienza con los test y luego se va al codigo.

Testing en C

Hay varias maneras:

- Usar la macro assert, no recomendado ya que aborta con el primer fallo y no da informes.
- Hacer tu propia libreria de testeos
- Usar frameworks de terceros

CuTest

Framework para realizar testeos, conviene usar este ya que es muy simple y lo suficientemente completo para el alcance de la materia. Corre todos los test, no aborta, y presenta mensajes mas detallados. Ademas, tiene mayor variedad de funciones, entonces mejora la calidad.

Terminos

- Test unitarios: Funciones que reciben por parametro un `CuTest *` e invocan alguna funcion de la libreria.
- Suite de test: Funcion que retorne un `CoSuite *` y tiene al menos un test unitaro. Engloba los test que pertencen a un modulo en particular.

  Nota: Tener un archivo separado para cada suite.

Unit test structures & patterns

- Behavior-Driven Development(BDD), Given-When-Then(Precondiciones-Determidado caso-Entonces)

Repo de materia con ejemplo: https://github.com/alejoaquili/c-unit-testing-example

Git hooks

Son scripts que Git ejecuta antes/despues de algun evento. Estan en `.git/hooks`, eventos

- Pre-commit
- Post-commit
- Pre-push
- etc.

Si falla, corta el commit.

Workflow GIT

Workflow basico de git para poder trabajar de manera ordenada y minimizar errores. La idea es tener un sistema de tracking de cambios de un set de archivos,

Branch

Representa un espacio de trabajo que contiene el codigo del repositorio y sobre la cual se pueden generar distintas versiones.

A su vez git tiene un buen sistema para unificar los cambios realizados en varias branches si se tiene una jerarquia correcta entre distintas ramas. Generalmente se tiene la ultima version funcional en main, mientras que en la branch develop se tiene lo ultimo de desarrollo(es la branch mas activa). Luego, se tienen las branch de develop que son las features, entonces se hace marge de una feature con develop primero y por ultimo de hace un merge con main. Comandos utiles:

- rebase
- stash
- cherry pick
- log
- reset

Practica 3: TP02

- Mantener un repo simple, no complicarse. Por ejemplo, las funcionalidades graficas.
- Cada modulo dependera del anterior, asi que hay que implementarlo secuencialmente.
- El administrador debe funcionar de forma trasparente, el contrato es siempre el mismo: devuelvo o pido memoria.
- Nos van a pedir tests. No deben ser pantalla en negro, debe imprimir lo que esta ocurriendo en todo momento.
- La shell es un proceso, va a haber que ofrecer un mecanismo para permitir procesos en foreground y en background. Con el `&` o cualquier otro caracter reservado.
- Notar que background y foreground son una abstraccion del front, basicamente reciben o no input del teclado.
- Que pasa si un proceso en background realiza la syscall `read()`. Esta es una situacion bastante sensible y si se tiene una implementacion con pipes es muy bueno. Pero se podria simplemente devolver un `EOF` si esta en background y listo(negarle el acceso al read).
- Para probar si anda bien todo lo de procesos es meter un break antes de la clase y crear 4 procesos que impriman constantemente por la salida. Si se imprime todo al continuar la ejecucion, entonces esta bien.
- Si no se ve nada por salida estandar, el test es invalido.
- Bajan nota si se hace esto, la lectura del teclado es un loop preguntando por mas caracteres. Esto valia en arqui, pero ahora no se puede hacer. Ahora el read debe ser bloqueante, no se debe consumir CPU. Muy facil de implementar con semaforos(una vez los hayamos implementados).
- La shell es la demostracion de que todo funcione, sino se ponen a ver el codigo, pero no es lo ideal.
- Una vez se llega a procesos, se puede comenzar a separar tareas para semaforos, comandos de la shell, etc.
- Para ver la cantidad de memoria disponible, chequear manual de pure64.
- La decision de que memoria organizar, depende de cada uno. Podemos manejar la memoria del kernel(Tomar analogia del bibliotecario) o no, tenemos la libertad de decidir.

Memory management

Tiene varias capas(de mas alto a mas bajo nivel):

- User-Space Allocator(address space disponible)
- Virtual Memory Manager
- Physical memory manager / Page Frame Allocator

Responsabilidades del memory manager

- Liberar memoria previamente asignada
- Asignacion **exclusiva** de memoria libre

Estas son las dos funcionales, pero la catedra agrega como no funcional la capacidad de consultar el estado, cuanta memoria esta asignada y cuanta esta libre.

Consideraciones para el TP

- 1:1, sin memoria virtual.
- Memoria para el MM se arma con el manual de Pure64 y la guia de building.
- Fragmentacion.
- Alineacion de memoria con el tamano de palabra con la arquitectura en la que estemos trabajando.

Buddy allocation

- Trabaja con un arbol binario.
- Se puede ver en el libro mencionado en la presentacion.

Practica 4: Context switch, scheduling y procesos

Importante mandarle al PIC que se recibio la interrupcion(EOI). Luego en conviene guardar el contexto de cada proceso en el stack y en el PCB guardar el stack de cada proceso.

Conviene al llegar al handler(que ya tiene los registros especiales en el stack) hacer `pushState` para guardar los registros de proposito general. Luego, al hacer `popState` se restauran todos los registros. Mandar al PIC el EOI despues del `pushState` pues sino se estaria pisando RAX.

Para hacer **context switch** con todo lo que armamos, es cuestion de cambiar el RSP a que apunte al stack del otro programa(que sufrio el mismo proceso) antes de `popState`.

Y cada stack esta en el bloque de memoria designado para cada programa por el memory manager, en el PCB se guarda la direccion de RSP.

Crear un proceso

Stack

Al crear el stack, vamos a mentir un poquito, vamos a rellenar el stack frame manualmente. Esto es para que el scheduler pueda menjar el stack de un proceso recien creado y el stack de un proceso que ya estaba corriendo.

Si el stack no estaba alineado, va a haber un problema, por lo tanto hay que rellenar para que el stack este alineado a la palabra de la arquitectura en la que estamos trabajando.

Tener cuidado, pues el stack crece hacia abajo, entonces, para hacer malloc hay que hacer algo como esto:

```
rsp = ptr + STACK_SIZE;
```

RIP

A `create_process` hay que mandarle algun argumento(son varios) que nos ayude a completar el RIP. Una opcion seria la direccion del `main` de dicho proceso, no tiene que ser exactamente `main`, pero tiene que ser de ese estilo.

> Tener cuidado con los argumentos que sean punteros como el `argv`.

Si no hay proceso ready?

Podemos disponer de un proceso cuya unica funcion es ejecutar `hlt`.

Practica 5: Sincronizacion

Lo vamos a necesitar no solo para semaforos, sino tambien para el wait() a los hijos. En esta clase vamos a ver como implementar todo esto, hay que resolver como entrar y salir de la zona critica cumpliendo las condiciones dadas en la clase(ver apuntes).

Las primitivas que tenemos que implementar, no es implementar un semaforo, sino que son:

- Enter region
- Leave region

Esto es porque para implementar un semaforo, hay que protegerlo a su vez con otro semaforo, entonces ahi entran las primitivas.

Dentro de la region critica

No se puede simplemente `cli` dentro de cada region pues dejaria de ser concurrente. Tampoco espera activa pues es lo que queremos evitar y porque se rompe facilmente la exclusion mutua, es debido a que el while necesita comparar una variable(condicion de carrera).

Otra es la alternacion estricta, pero solo se puede tener entre dos programas o que uno termine, entonces el otro se queda esperando para siempre pues depende del otro.

Tambien esta la solucion de Peterson, similar al anterior, pero agrega un flag.

Soluciones

Spin lock

Hay un poquito de espera activa, pero como dura 1-4 quantums se asume que es correcto. Implementa `adquire` y `release`, son las que tenemos que implementar en el TP.

> **Nota**: MUY MAL no delver el SPIN LOCK antes de bloquearse.

Teorica 1: Introduccion

Sistemas operativos

Primero planteemos que ocurriria si no hubiese un sistema operativo, entonces, `printf` no funciona, no tiene sentido. No hay drivers, no hay nada definido y no existe el SO que puede proveer el servicio de imprimir en pantalla.

En conclusion, el sistema operativo provee servicios(resuelve problemas) a nivel de llamar una funcion que haga algo, abstrayendo los detalles(no me importa en que hardware lo este corriendo).

Provee un conjunto obstracto y limpio de los recursos, administra estos recursos. Se requiere que el hardware tenga por lo menos dos modos de operacion(con distintos privilegios) para que funcione correctamente.

Oculta el hardware y ofrece una interfaz limpia, elegante y consistente al programador, las abtracciones son una de las claves para comprender SOs.

Administrador de recursos

Que pasa si 2 o mas procesos intentan imprimir en la impresora?

El SO es un ente centralizado que agrega cada impresion a un queue y garantiza que el acceso al recurso impresora sea correcto.

Administrar recursos incluye **multiplexar** estos recursos, mediante alguno de los siguientes metodos:

- Tiempo
- Espacio

Multiplexar trata de la distribucion de un recurso(temporal o espacialmente), por ejemplo, dividir el auditorio de SDT en clases mas chicas(espacial).

Revision hardware

Procesador

Un registro importante es el RFLAGS.

Al iniciar el hardware, el stack estara vacio y el IP apuntara a la direccion de nuestro programa, pero primero hay que determinar cual es esa direccion.

Multithreading Mantener el estado de 2 threads e intercambiar entre ellos rápidamente cuando sea necesario. No es paralelismo.

Multicore Replicar los núcleos independientes. Pueden soportar múltiples threads.

> **Nota**: Ley de Moore es una aproximacion bastante acertada, pero llegamos a una limitacion de temperatura. Hoy en dia simplemente se replican los nucleos(muchos nucleos en un mismo procesador).

Memoria

Idealmente la memoria debería ser:

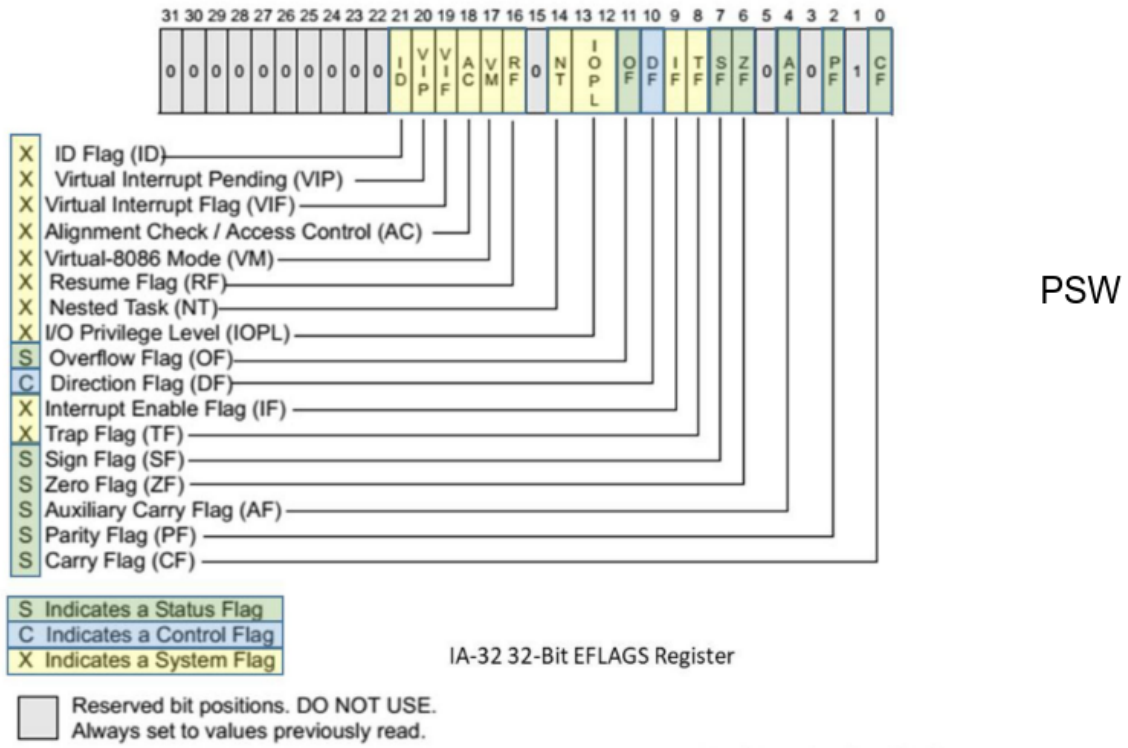

Figure 4: RFLAGS

- Extremadamente rápida (más que la ejecución de una instrucción)
- Absurdamente grande (para albergar a todos los procesos juntos)
- Asquerosamente barata

Obviamente esto es una utopia porque en la practica no se cumplen estas condiciones, por lo tanto, surgue una jerarquia de memoria(ver figura).

Dispositivos I/O

Un dispositivo consta de 2 partes:

- Controlador (abstracción)
- Dispositivo

Hay 3 formas de IO:

- Busy waiting(polling), en este caso, el cpu pregunta(es un desperdicio)
- Interrupción
- DMA (Direct Memory Access)

Proceso de interrupcion(?):

1. El driver le indica al controlador qué hacer. Este inicia la operación en el dispositivo.
2. El controlador señaliza al controlador de interrupciones (CI).
3. Cuando el CI está listo (otras interrupciones) setea el pin correspondiente en el CPU.
4. El CI escribe el número de dispositivo (vector de interrupciones) en el bus.

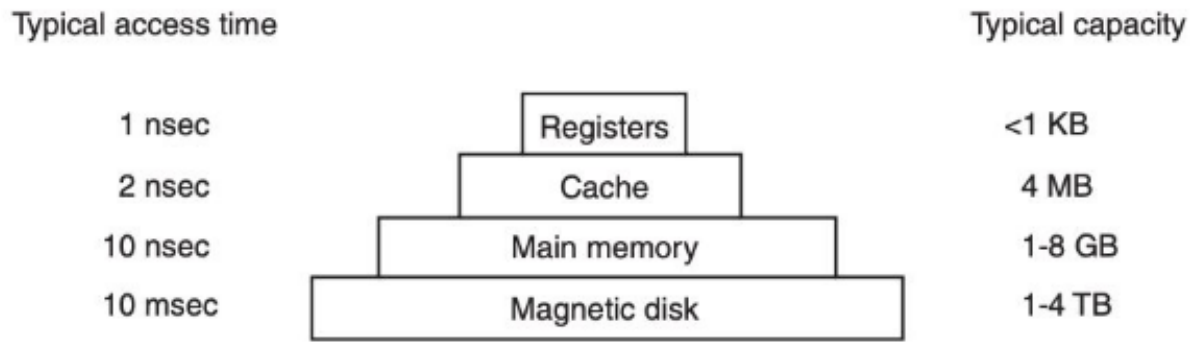

Figure 5: Jerarquia de los tipos de memoria

Booteo

- La placa madre posee un programa llamado BIOS (Basic I/O System)
- Flash RAM, no volátil pero actualizable por el SO
- La BIOS posee instrucciones para interactuar con la pantalla, teclado y disco

Proceso de booteo(una vez se determina cual es el dispositivo de booteo:

1. Chequear cuánta memoria tenemos y si ciertos dispositivos como el teclado están instalados y responden correctamente.
2. Escanea buses PCIe y PCI en busca de dispositivos instalados.
3. Determina el dispositivo de booteo.
4. Se carga el primer sector en memoria y se ejecuta
5. Este sector suele leer una tabla de particiones y se le carga un segundo sector
6. Se carga en memoria el SO y se ejecuta.
7. El SO consulta a la BIOS los dispositivos conectado

Tipos de SO

Ordenados de mas grandes a mas chicos.

- **Mainframe**: Mucho almacenamiento y memoria. Orientado a muchos procesos. Soporta trabajos por lotes, transacciones y timesharing. OS/390, tendencia a UNIX.
- **Server**: Computadoras personales o incluso mainframes. Impresoras, archivos o webs. Solaris, FreeBSD, Linux y Windows Server.
- **Multiproceso**: (Necesaria la separación)
- **Personal Computer**: Proveer soporte a un único usuario. Linux, FreeBSD, Windows y Apple's OS X.
- **HandHeld Computer**: Tablets y smartphones. Android y iOS
- **Embedded**: Electrodomésticos y teléfonos antiguos. ROM, no permite la instalación de aplicaciones. Embedded Linux, QNX y VxWorks.
- **Sensor-node**: Redes de nodos. Poseen CPU RAM y ROM. Sensan diferentes condiciones. Conexión inalámbrica. TinyOS

- **Real Time**: Necesitamos que una acción ocurra en un momento específico. hard vs soft. Industria, aviónica, milicia y multimedia. eCos y FreeRTOS
- **Smart Card**: Pagos electrónicos. Del tamaño de una tarjeta de crédito. Restricciones de potencia y memoria muy altas. Obtienen energía por inducción o al conectarse a un lector.

System calls

Como vimos en arqui, las syscall buscan:

- Proveer un conjunto abstracto y limpio de los recursos (hardware) -> entender qué hace el SO
- Administrar estos recursos -> transparente para el proceso
- Se puede ver como una simple llamada a función que además cambia a modo kernel
- El mecanismo para realizar una system call depende de la arquitectura y debe expresarse en assembler.

POSIX es basicamente la API de linux, en windows se llama **win32**, nosotros vamos a ver el grupo principal de syscalls definidos por POSIX.

> **Nota**: Malloc puede derivar de una syscall, pero en realidad solo es necesario realizar una syscall cuando se acaba la memoria disponible para pedirle mas memoria al kernel.

Read

Para verificar que la primera vez que se ejecuto `read` levanto todos los datos posibles, se corre una segunda vez, si devuelve 0 significa que no hay mas nada para leer (si le pedis 1024 y te da 1000 y despues 0, entonces el archivo es de 1000 bytes) y la lectura fue exitosa. Sin embargo, si la segunda vez devuelve mas que 0(en el caso de leer un archivo) entonces no se leyo todo y hay que volver a hacerlo.

Waitpid

Hace que el proceso espere a que termine el proceso con el pid especificado. Durante la espera el programa se queda bloqueado y no ocupa ciclos del procesador, pero si memoria. Se entera el parent porque el SO termina la syscall.

Por otro lado, el SO se entera de que termino por la llamada a `exit()`(la cual es una syscall) del child. Se fija quien es el parent y si el mismo esta esperando lo notifica.

> **Nota**: Hay mas syscall en la T1.

Kill

Manda una senal que alerta a otro proceso, simplemente le avisa, pero no necesariamiente lo mata. Ahora, si envio la senal -9 ahi si termina. Cada programa tiene un handler para las senales, entonces los handlers se general en runtime y se setean con syscalls. Esto permite que un programa en vez de terminar muestre una ventana que pregunte al usuario si esta seguro de que quiere terminar el proceso.

El SO hara que el IP cambie a apuntar la funcion del handler, lo hara sin importar si la funcion no esta en su lugar y la va a ir a buscar. Implementa **zero trust**, se va a asegurar que el programa termine si se envia la senal de terminacion.

Fork

Mis palabras:

> Se crea un proceso identico que sigue la ejecucion en el momento que retorna.

Asi es, **KISS**. Es un ejemplo de las varias funciones no siguen lo usual(exec(no retorna), exit(no retorna), etc.): llamo una vez y retornan una vez.

A donde va la salida estandar

Ejecutan write en fd 1, no les importa a donde apunta el fd, eso depende del SO. Algunos ejemplos:

- `ls` -> terminal,
- `ls | grep` -> pipe
- `ls > foo.txt` -> archivo
- `ls > /dev/null` -> dispositivo

Estructura de un sistema operativo

Monolithic systems

Toda la funcionalidad del SO esta en un solo binario, toda funcion tiene visibilidad del resto de las funciones. Esto es eficiente, pero complejo. Un error en cualquier funcion hace fallar el sistema operativo completo. Ademas, soporta **shared libraries** o **DDLs**.

Presenta una filosofia de estructura, ver figura. El TP arqui es monolitico(como el TPE de SO), el user space esta separado, pero no es parte del SO.

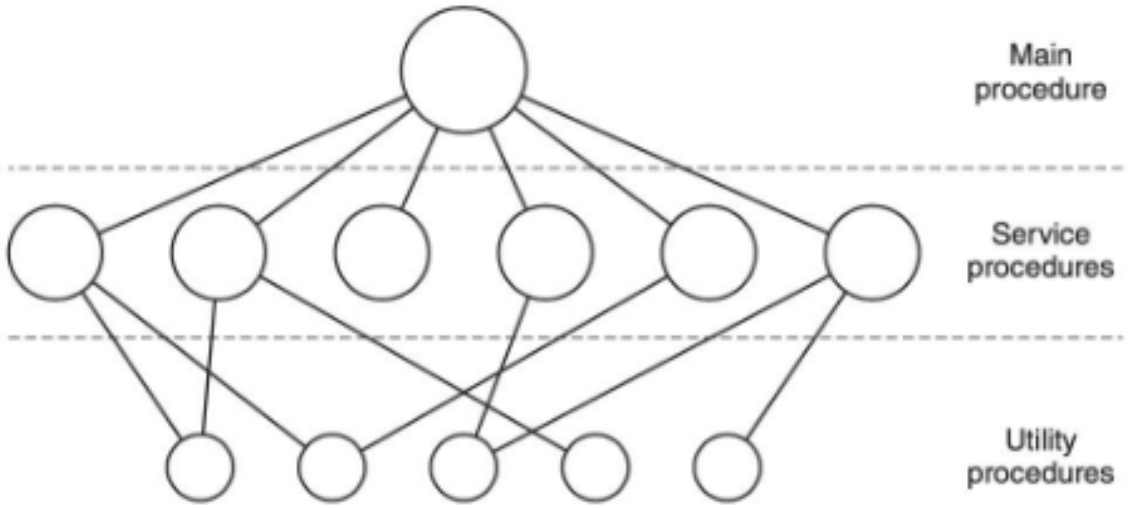

Figure 6: SO monolitico

Nota: Es muy usado, por ejemplo, Windoes, FreeBSD, Linux, etc.

Microkernels

La idea es dejar los componentes esenciales en el kernel(lo minimo), mientras que el resto se va a correr como clientes del kernel(en user-mode). Los drivers corren como un proceso mas, por lo que un error fuera del kernel no hace fallar al SO. Tiene alta confiabilidad(Real time), implementado por Integrity, PikeOS, MINIX, etc.

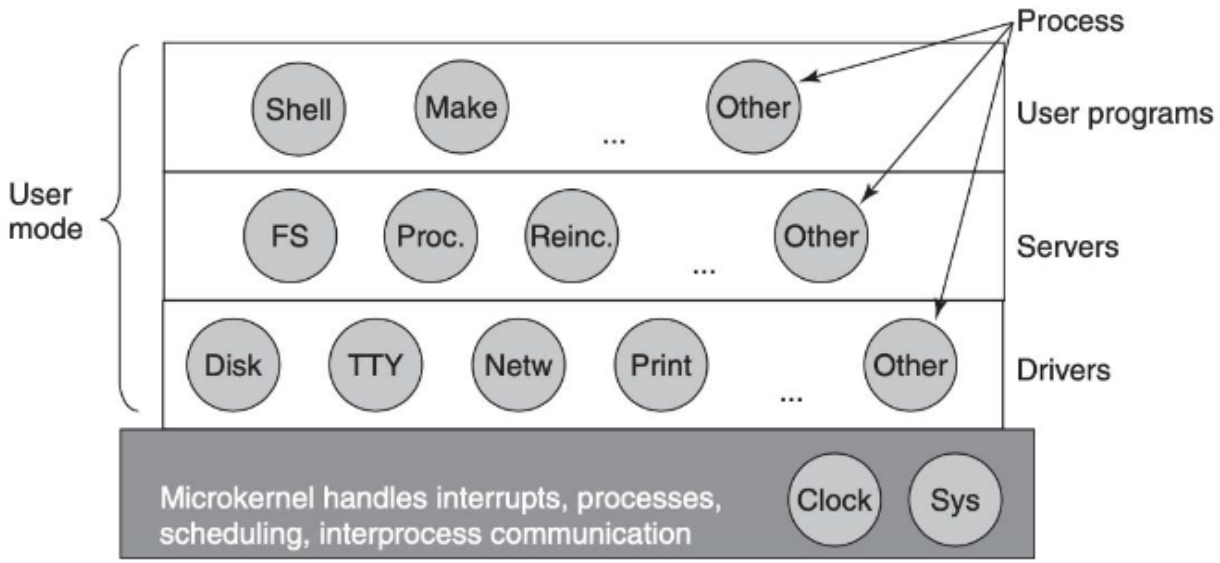

Figure 7: SO microkernels

Un ejemplo, MINIX, presenta un **reincarnation server** el cual es el padre de todos los drivers y servers, chequea constantemente de controles que esten todos vivos. Limpia los drivers y servers muertos, y chequea una tabla para ver que hacer(generalmente los reinicia).

Mecanismo vs politica La separacion entre mecanismo y politica es una buena estrategia de programacion pues permite reducir el tamano del kernel y separar areas diferentes. Colocar el mecanismo en el kernel y la política por fuera, por ejemplo, asignar prioridades a los procesos(por fuera del kernel) y ejecutarlos según estas prioridades:

- Elegir al proceso de mayor prioridad (mecanismo) puede estar en el kernel
- Asignar las prioridades (política) puede estar por fuera del kernel.

Nota: Hay implementaciones hibridas que combinan ambas.

Virtual machines

Tradicionalmente Mail server, FTP server, Web server, etc corren en computadoras separadas, lo cual lo hace popular para Web hosting: servidor dedicado vs. compartido - costo y flexibilidad Tambien presentan utilidad para uso personal. Para que funcione, el CPU debe ser virtualizable: SO corriendo en user mode

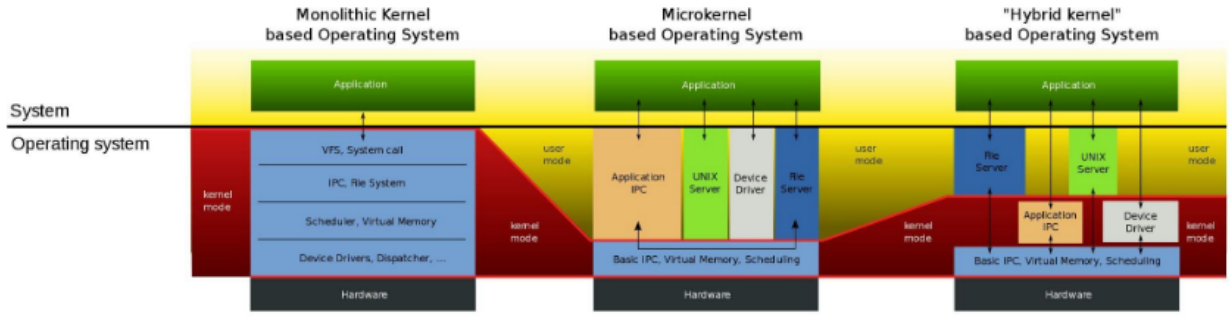

Figure 8: Monolitico vs Microkernels

ejecuta instrucción privilegiada, es necesario que haga un TRAP al virtualizador para emularla en software. Pentium ignoraba estas instrucciones aunque existían intérpretes como Bochs (bajo rendimiento).

No es exactamente lo mismo que un emulador de hardware, pero si hace creer a los SO dentro de los VM que son su propia maquina con su propio CPU, RAM, etc. Son maquinas separadas, lo cual lo hace conveniente por cuestiones de costo y seguridad.

Hay diferentes tipos de maquinas virtuales, tambien conocidos como **hypervisors**:

- **Virtual machine monitor o type 1 hypervisor**: Se ocupa de proveer los servicios necesarios.
- **Type 2 hypervisor**: Tiene un SO por debajo que se encarga de abstraer los procesos, file system, etc. QEMU y BOCHS.
- Sistemas híbridos.

Es mas eficiente y rapido el tipo 1 debido a que tiene menos capas e interactua directamente con el SO. El tipo 2 tiene ya una capa extra.

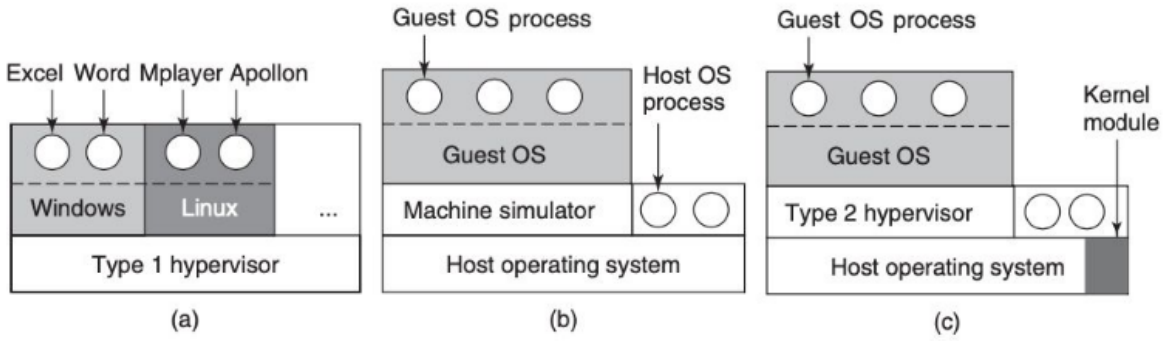

Figure 9: Tipos de VMs

Teorica 2: Procesos

Todo software se organiza en **procesos secuenciales** o simplemente **procesos**, hay que diferenciar bien entre:

- **Proceso**: Abstraccion de programa en ejecucion (si ponemos esto en el examen va a estar bien).
- **Programa**: Almacenado en disco, no hace nada.

  Nota: Esto puede ser pregunta de examen, como todo.

Que podria ocurrir si tenemos un programa que corre dos veces? Lo que ocurre es que tengo dos procesos y un programa.

Para que un programa pueda ser ejecutado, se debe cargar el codigo en memoria para poder convertirse en proceso. Posee sus propios registros y variables, habra un stack por cada proceso.

> Desde el punto de vista del proceso, el es dueno del CPU.

Esto se debe a que el proceso no se tiene que preocupar por los otros procesos, por ejemplo, saber si el CPU esta ocupado o si otro lo quiere usar. Por esto se llama proceso secuencial, para el proceso es una instruccion atras de la otra, no percibe los bloqueos de tiempo.

Pseudo concurrencia

Periodo de tiempo muy corto, llamado **quantum** o **time slice**, en el cual se corre un proceso antes de ser interrupido. Estos periodos son tan cortos que son imperceptibles para el usuario.

Tambien se lo suele llamar **pseudo paralelismo**, no es concurrencia porque en realidad no esta ocurriendo todo al mismo tiempo. Para tener paralelismo real se necesitan de multiples nucleos compartiendo memoria fisica, eso es **paralelismo real**.

No hay que pensar en los **switches**, es decir, que no notamos los saltos entre procesos dentro del CPU. Este switch no es uniforme ni reproducible ya que no puedo hacer supuestos sobre tiempo de ejecucion.

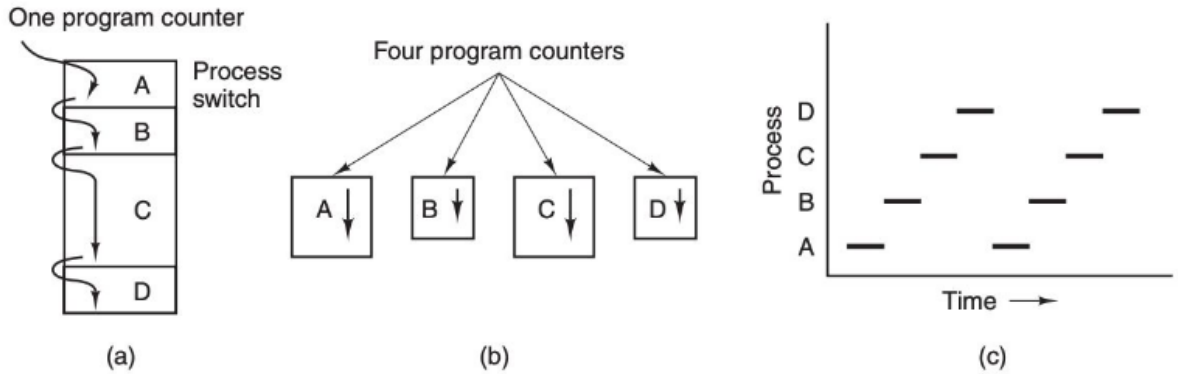

Figure 2-1. (a) Multiprogramming four programs. (b) Conceptual model of four independent, sequential processes. (c) Only one program is active at once.

Figure 10: Modelo de proceso

Reflexion

Cuando se tiene un bug, es deseable que sea facilmente replicable, que sea aislable. Luego, si se tiene dos programas con memoria compartida, donde el proceso 0 es el que initializa la variable compartida y el 1 la

utiliza, por los switches, no puedo suponer que 0 se ejecutara antes de 1.

Este es un supuesto de tiempo de ejecucion que no es correcto hacer, entonces, a veces funciona y a veces no, dependiendo de como se ejecuto el switch, haciendo que sea dificil de replicar. La idea es programar de manera que seamos inmunes al orden de ejecucion del SO.

> **Nota**: El context switching es costoso para el CPU.

Ademas, la responsabilidad principal del **scheduler** es repartir de manera homogenea el tiempo de ejecucion dentro del CPU de cada proceso, la manera mas simple es haciendo una lista circular(0123012301230123...) e ir ejecutando cada programa por una cantidad de tiempo determinada antes de ir al siguiente(se conoce como **round rolling**).

> Siempre conviene hacer algo funcional(basico) primero(no tiene que ser eficiente ni lindo) para poder empezar con algo, para asegurar que es correcto y usar de referencia. Permite hacer experimentos y verificar que todo funcione todo primero.

Creacion de procesos

Un proceso tiene una vida en el SO: nace -> ejecuta -> muere. La creacion de un proceso requiere de un SO, cualquier cosa que no este siendo creada por un kernel no es un proceso, por ejemplo, una syscall. Puede tener el mismo codigo y hacer lo mismo, pero no sera un proceso.

Casos que requieren de un proceso:

- Se inicia un SO(Daemons).
- Proceso solicita la creacion de otro(Script bash y comandos UNIX).
- Usuario solicita crear un nuevo proceso(Terminal, GUI).
- Inicio de un trabajo por lotes.

Siempre es un proceso creando otro proceso, entonces, de donde sale el primer proceso?

El sistema operativo es la respuesta correcta, no el "proceso" sistema operativo, pues el SO provee la abstraccion de un proceso, pero el por si mismo no es un proceso. De igual manera que el scheduler no es un proceso. La creacion del primer proceso es mas "artesanal", es la excepcio a la regla, luego los demas van a ser creados como se describio antes.

Win32

Para crear un proceso se llama a `createProcess()` el cual tiene 10 parametros:

- El programa a ejecutar
- Parámetros
- Argumentos
- Atributos de seguridad y control
- Prioridad
- Ventana a asociar con el proceso

Hay mas de 100 syscalls extras para administrar y sincronizar procesos, en ambos casos(UNIX y Win32) los procesos creados tienen su propio espacio de direcciones, diferente al padre.

UNIX es mucho mas elegante ya que tiene mucho menos syscalls y no cambiaron los prototipos de las funciones. Basicamente, el **core** se ha mantenido igual durante un monton de tiempo.

Terminacion de procesos

Un proceso puede terminar:

- Salida normal(voluntaria)
- Salida por error(voluntaria)
- Error fatal(involuntario), se envia una senal y el comportamiento por defecto es terminar(no se puede cambiar el manejador).
- Muerto por otro proceso(involuntario), por senal, pero si se puede cambiar manejador.

> **Nota**: Exit es una syscall. Si necesito una syscall para iniciarlo, entonces necesito una syscall para terminarlo.

Jerarquia de procesos

Un proceso y su creador preservan una relacion, generalmente padre / hijo. No significa que un proceso sea mas importante que otro, sino que se trata de una relacion entre procesos. En UNIX, el proceso padre de todos los procesos es `init`, es la raiz del arbol. Si se mata a un proceso padre, se manda una senal al padre y muere, pero si quiero matar a los hijos voy a tener que mandar una senal a cada uno, no mueren en cascada.

> **Nota**: En nuestros TP02, la shell va a ser nuestro `init`, aunque podriamos usar otro proceso, suele ser la shell.

Grupo de procesos

Un padre y sus hijos se denominan un grupo, esto tiene ventajas como poder mandar una senal a un grupo entero o atencion de senales independiente. En general, grupo de procesos que esta leyendo del teclado tiene un solo miembro. Para `init`, se puede secir que es analogo a la clase `object` en Java.

Win32

No existe jerarquia de procesos, el padre recibe un handler para manejar al proceso. El handle se puede transferir a otro proceso.

Estado de procesos

Un proceso puede tener uno de 3 estados fundamentales:

- Running(usando el CPU en ese instante)
- Ready(ejecutable; detenido para dejar que se ejecute otro proceso)
- Blocked(no se puede ejecutar hasta que ocurra algun evento externo)

En general, los primeros dos podrian no implementarse, se podria dejar simplemente como `runnable` y listo, no es siempre necesario llevar registro de los otros dos. Blocked, por otro lado, si debe tenerse en cuenta.

Que un proceso este bloqueado(syscall implicita) significa que no se le da tiempo en el CPU hasta que ocurra la condicion esperada, esto se ve cuando se espera el input de un fd pues se usa la syscall `read()`.

> **Nota**: Se puede verificar el estado de un programa con `strace`.

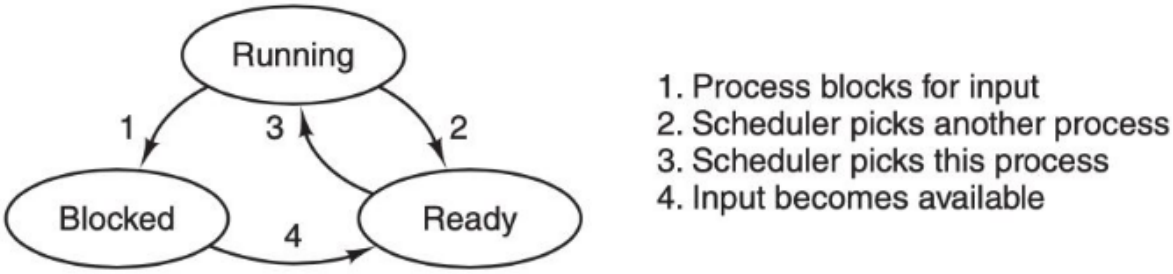

Figure 2-2. A process can be in running, blocked, or ready state. Transitions between these states are as shown.

Figure 11: Ciclo de vida de un proceso

Transcisiones

Clasico ejemplo de **ejercicio de final**:

- **Evento que pasa de running a blocked**: waitpid
- **Evento que pasa de blocked a Ready**: exit
- **Evento que pasa de running a ready**: interrupt del timer
- **Evento que pasa de ready a running**: interrupt del timer(saca a un proceso y agarra uno en ready y lo pone en running)

Pregunta de parcial: Se rompió el timer. No existe otra fuente de int. ¿puede el SO volver a tomar el control?

Si, ya que estan las interrupciones de software, las syscall. La ejecucion de una syscall no necesariamente le devuelve el control al proceso. Una vez le damos el control al kernel, no tenemos garantia de lo que va a pasar.

Pregunta de parcial: Porque no hay transicion de ready a blocked y de blocked a running?

La transicion de running a blockeado ocurre de manera sincronica, mientras estando ready no puedo pasar a blocked simplemente porque no se esta ejecutando, no tendria ningun sentido la transicion. No se puede realizar division por 0 o inv. opcode estando ready.

> Para desbloquear el proceso tiene que primero ser desbloqueado para ready por el so luego de algo externo, entonces no pasa nunca de blocked a running sin un intermedio en el que se pone en ready mientras corre algo más.

Implementacion de procesos

El SO mantiene una tabla de procesos donde almacena informacion privada de cada proceso. Es una tabla de structs, el struct pueden ser tan complejo como uno quiera. Cada entrada se denomina Process Control Block(PCB). Es la informacin necesaria para pasar de ready -> running -> blocked.

Si el proceso termino, a quien le puede interesar el valor de retorno de dicho proceso?

Al padre, al SO no le va a importar. Lo unico que le importa al SO es si el proceso termino. El padre puede conocer el exit status de su hijo mediante el PCB, pero no directamente pues es del SO. Entonces, se usa una syscall en particular, se podria usar la syscall `wait()`. Recien cuando hace `wait()` el padre, el SO borra el PCB.

Entonces, si el hijo termino, pero no se borro el PCB, se coloca el estado **zombie**. Esta en ese estado desde que ejecuta `exit()` hasta que alguien ejecute `wait()`. Si muere el padre, los hijos quedan en estado **huerfano** y son adoptados por `init`, de manera que rapidamente dejan de ser huerfanos, es decir, esta el estado, pero dura muy poco. Entonces, para evitar que se acumulen los huerfanos, `init` hace `wait()` a todos estos procesos.

> **Nota**: En el TP02 los registros se guardan en el stack(el mismo del proceso que estoy frenando).

Cuando se interrumpe un programa, se crea un nuevo stack para el scheduler/interrupt handler que no sea accesible por el programa que acabo de ejecutar. No es buen diseno que sea el mismo stack que el programa anterior pues ese stack sera usado por funciones del kernel y eso no es bueno por cuestiones de seguridad(uno con el stack puede ver un monton de datos, como valores de variables locales, etc.).

Porque necesita uno por proceso el kernel?

Porque hay funciones que por ahi no retornaron de un proceso, no haria falta un stack por proceso si se hace todo y se popea todo del stack antes de volver al programa. Sin embargo, las funciones que bloquean como`wait()` no retornan inmediatamente y todas esas funciones tienen su direccion de retorno en el stack del kernel, por eso hay un stack del kernel por proceso.

Modelado multiprogramacion

Caso:

- Un proceso usa el CPU el 20% del tiempo, el resto esta esperando por I/O(mientras corre).
- Entonces 5 procesos mantendrian el CPU al 100%.

Realista?

NO. Sera alrededor del 70%, por ese motivo la multiprogramacion es muy efectiva. Permite aprovechar el hardware tanto como se pueda.

Teorica 3: Inter process communication(IPC)

Los procesos cooperativos se distribuyen una tarea entre todos, mientras que los independientes hacen todo ellos.

Motivacion:

- Compartir informacion
- Acelerar computacion
- Modularidad
- Conveniencia

Hay dos modelos fundamentales:

- Memoria compartida
- Pasaje de mensajes

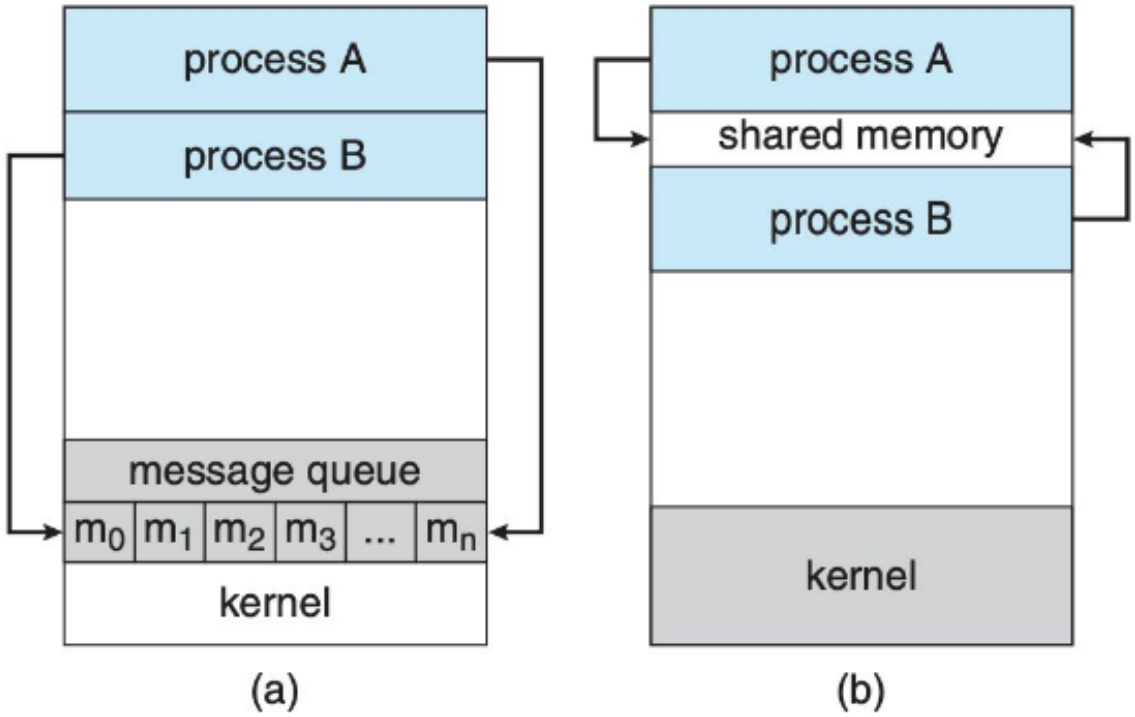

Figure 3.12 Communications models. (a) Message passing. (b) Shared memory.

Figure 12: Dos modelos fundamentales

Memoria compartida

Los procesos acuerdan compartir memoria fisica(para ello hay que hablar con el SO), generalmente no requiere de syscalls para acceder a la informacion. La organizacion de la informacion y sincronizacion es determinada por los procesos. Surgen problemas de cache coherency al aumental los nucleos.

Meter al kernel en el medio de la memoria compartida no tiene ningun sentido, no hace falta acceder a la memoria con un fd, es un gasto de syscalls. Lo que hay que hacer es usar el puntero a la direccion de la memoria.

Para que dos procesos accedan a la misma porcion de memoria se utiliza el nombre, el cual es una ruta en el filesystem (no es absoluta, en el manual aparece cual es la absoluta).

Hay que tener cuidado al manejar los punteros de la memoria compartida, pues no es el mismo para diferentes procesos por mas que sea la misma porcion de memoria. Por lo tanto, para posiciones relativas conviene leer el offset y compartir ese dato entre procesos, no el puntero en si.

Pasaje de mensajes

En este caso el kernel interviene, impone una sincronizacion implicita, no hace falta hacer polling sino que se bloquea el proceso. Es más simple en sistemas distribuidos, util para pequeñas cantidades de información y tiene un tamaño de mensaje fijo o variable.

Operaciones:

- send(message)
- receive(message)

Es necesario un canal de comunicación (memoria, bus, red)

Sincronizacion

`send` y `recieve` pueden ser bloqueantes o no bloquearse. Hay varias opciones:

- **send bloqueante**: hasta que llega a la casilla / proceso.
- **send no bloqueante**: resume inmediatamente.
- **receive bloqueante**: hasta que hay mensaje disponible.
- **receive no bloqueante**: recibe mensaje o null.

Buffering

Los mensajes residen en en un buffer

- **Capacidad 0**: no buffering -> send debe bloquear.
- **Capacidad acotada**: send debe bloquear si se agota el espacio.
- **Capacidad no acotada**: send no bloquea.

  No es conveniente usar syscalls en los programas que se bloquean.

Va a haber que desglozar el programa en dos modulos distintos si tiene que hacer cosas mientras espera a la informacion(no bloqueante).

Ejemplos de IPC

Files

Un archivo es una forma de comunicar procesos que se adapta bastante bien al modelo de memoria compartida, en este caso si interviene el kernel, pero el kernel no indica como almacenar las cosas.

Generalmente no se lo menciona como memoria compartida, pero para Ariel si se puede interpretar de ese modo.

Pipes

Mecanismo muy "estrecho" de pasaje de mensajes, solo en un sentido. Fue uno de los primeros mecanismos de IPC en UNIX, muy simple y muy usado. Tiene un buffering acotado y la sincronizacion se basa en `send` bloqueando al llenarse el pipe, mientras que `recieve` bloquea hasta que haya algo.

Hay dos formas de definir pipes:

- **Ordinarios/Anonimos**: No tienen nombre, significa que no tienen una entrada en el file system.

- **Named pipes**: Tienen nombre, permite comunicar dos procesos ordinarios.

Si no tiene un path(anonimo) como puede el otro programa encontrar el pipe?

Basicamente, se hereda, pues se mantienen los fd al hacer `fork()`. Pero no se limita solo a padre-hijo, sino que en UNIX se podria acceder a los fd correspondientes leyendo del directorio `/proc`.

Files vs. pipes

Ventajas de los pipes:

- Se eliminan automáticamente (anónimos)
- Cantidad arbitraria de información. Si el pipe se llena se bloquea el que envia, en cambio un archivo tiene un limite de espacio.
- Ejecución paralela
- Sincronización implícita

Pipe tambien es mas rapido, pues en caso de que haya un disco(podria no haberlo en UNIX) es muy lenta la escritura a disco. Se puede lograr practicamente lo mismo, pero al final del dia es mucho mas conveniente usar pipe.

Sockets

Permite comunicar procesos a traves de una red, es bidireccional(full duplex). Un socket se identifica con una IP y un puerto, generalmente sirve para una aplicacion cliente - servidor.

FIFO vs Unix Domain Sockets

Los pipes que usamos son simples, tienen una sola direccion. Los sockets se pueden usar por muchos procesos a la vez, tienen un hilo escuchando conexiones entrantes y apenas llega una comunicacion se crea un socket paralelo para la comunicacion, pero eso no se muestra, desde afuera parece un solo socket(transparente para el cliente).

Como la radio de un puerto, todos van al canal 16, pero luego prefectura los manda a otro canal y libera el 16, la unica diferencia es que hay un canal individual para cada embarcacion en caso de sockets.

> **Nota**: Las signals son async, ya que el programa que recibe una signal no esta esperando la signal, sino que esta corriendo y es interrumpido.

Signals

Basicamente como kill es una syscall, el SO se encarga de buscar el proceso al que se le envio la signal y en base a la entry en el SO del programa se llama al handler. Pero el handler no lo llama el SO, sino que modifica el stack y los punteros del programa receptor de la signal para que la proxima instruccion que ejecute sea la del handler(el handler se corre en el contexto del programa).

No hay signals anidadas, pero tampoco es que no se recibe la signal. Lo que ocurre es que se recibe la signal y se espera a que termine el handler, una vez termina el SO procesa la otra signal. No hay un concepto re `ret` en un handler, el programa se interrumpte y luego de terminar el handler continua la ejecucion, pero no tiene porque hacer algo. Se podria definir una variable que se modifique en el handler para notificar al programa que este se ejecuto, pero nada mas.

Race condition

Una situación en las que dos o más procesos están leyendo o escribiendo datos compartidos y el resultado final depende de quién corre en cada momento.

No es sinonimo de estado inconsistente, sino que el resultado depende de la ejecucion. Es muy feo debuggear programas donde ocurren condiciones de carrera.

Esto se soluciona con una especie de patron de sincronizacion, es decir, el programa que escribe tiene acceso exclusivo y una vez termina libera el acceso exclusivo. Esto no impide que el escritor sea interrumpido, pero si hay garantia que ningun otro proceso podra escribir al mismo lugar. No es inmune al **context switch**.

Se ejecuta de manera atomica uno y despues se ejecuta otro.

Region critica

Es una parte de un programa, la parte que accede a la memoria compartida.

Mecanismos de sincronizacion

- **Semaforos**: El wakeup waiting bit se plantea(Dijkstra - 1965) como entero representando la cantidad de wakeups disponibles, el cual llamaremos semaforos. Tiene dos operaciones: **down** y **sleep**.

  Nota: Mutex es basicamente **mutual exclusion**.

Si dos quieren acceder al mismo semaforo, el primero que llega "gana el semaforo". Es una especie de race condition, pero no importa porque no afecta el resultado del problema.

El semaforo mutex es solo para proteger el acceso a la zona critica, mientras que el otro semaforo simplemente bloquea alguno de los dos procesos. Ver problemas clasicos de los apuntes.

Deadlock

Un conjunto de procesos está bloqueado (en estado de deadlock) si cada proceso del conjunto está esperando un evento que solo puede causar otro proceso del conjunto.

Teorica 4: Threads

Tradicionalmente un proceso tiene su espacio de direcciones y un unico hilo(thread) de ejecucion.

Existen casos en los que es muy util tener multiples hilos en un mismo espacio de direcciones corriendo de forma pseudo paralela como si fueran procesos separados, salvo por el espacio de direcciones. Los threads comparten el codigo, el heap, pero no stack, esa es la principal distincion entre dos threads y dos procesos. El stack no se comparte porque tiene informacion de la ejecucion, el nivel de anidamiento y las variables locales, entonces, usar el mismo stack no tiene sentido.

Un proceso solo ejecuta una instruccion en particular en cualquier momento, no confundir esto con la nocion de threads. Todos los hilos tienen el mismo codigo, heap y variables globales. En cuanto al stack, no pueden tener el mismo, es privado a cada hilo.

La motivacion es poder correr muchas actividades simultaneas, algunas bloqueantes. Desglosar la situacion:

- Aumenta el uso del CPU

- Simplifica la programacion
- Aprovecha arquitecturas con multiples CPUs

Tiene la misma nocion de modelo de procesos, donde se pensaba en procesos secuenciales, pero con el agregado de que comparten un espacio de direcciones.

Ademas, son mas 'baratos' pues se crean y se destruyen, y son hasta 10-100 veces mas rapido que un proceso. Un thread tambien se puede reutilizar, cambiando su funcion.

Un ejemplo seria el web server, un thread es el **dispatcher** que recibe conexiones entrates y delega la atencion de una conexion a otro thread, cada conexion termina con un thread individual.

> **Nota**: Como simular threads con solo procesos? Con memoria compartida.

Como seria un servidor sin threads?

- Syscalls no bloqueantes.
- Almacenar el estado de cada pedido para retomarlo cuando llegue la pagina del disco, esto es muy costoso.
- Se pierde la nocion de computacion secuencial.
- Estamos simulando threads **the hard way**.

Esto es basicamente modelar una maquino de estado finita, es paralelismo de manera que estan ocurriendo cosas de forma paralela, por ejemplo, el disco busca algo en cache mientras se corre codigo.

En las syscall no bloqueantes, se necesitan interrupciones para avisar al proceso que la syscall termino(por ejemplo, el read), debido a que sino el proceso no tiene manera de saberlo(sacando polling).

> Es mucho mas facil programar cuando se bloquea. No es recomendable usar syscalls no bloqueantes.

Modelo de threads

El modelo de procesos esta basado en 2 conceptos independientes:

- Agrupacion de recursos(PID, FD, pointer to text segment)
- Ejecucion(Register, program counter, CPU time used, process state)

Proceso vs. thread

Un thread esta contenido en un proceso, pero un proceso agrupa recursos mientras que thread es una entidad planificada(scheduled) para ejecucion en el CPU. Los threads permiten multiples ejecuciones en el mismo conjunto de recursos.

Los threads comparten el espacio de direcciones, por tanto, un thread podria modificar informacion mientras otro la esta leyendo, podria ocurrir un **race condition**. No existe mecanismo de proteccion por el kernel ante esta situacion, esto se debe a que es problema del programador, como el caso de memoria compartida. El kernel no se puede interponer en cada acceso a memoria. Como mucho el kernel puede tener memoria virtual y configurar el hardware para que me alerte con una interrupcion cuando se esta intentando acceder a memoria sin permisos.

El **yield** basicamente termina el quantum del thread antes, permitiendo que el scheduler le de el control a otro thread. Por ejemplo, si libero algun semaforo conviene hacer un yield inmediatamente para que otro

thread lo pueda usar. Basicamente, ayuda a la cooperacion entre procesos.

Implementacion en espacio de usuario

- El kernel desconoce de su existencia
- Desde la perspectiva del kernel son procesos con un unico thread
- Provee soporte para threads en caso de que el kernel no los provea
- Se implementan como una libreria

Esta no es la implementacion mas comun. Esta implementacion busca complementar kernels que ya de por si no tienen una implementacion de threads, de manera que se implementa como libreria.

Cada proceso necesita su tabla de threads privada, almacenada en el user space por cada proceso.

> Para librerias como errno, las partes que tienen permisos de escritura tienen una copia en cada proceso o al menos es COW. El resto del codigo que sea soloo lectura se lee del mismo lugar por todos los procesos para que sea mas eficiente.

En la implementacion con el kernel, si un thread se bloquea, para un mismo proceso, el resto puede seguir corriendo pues el kernel tiene conocimiento de sus existencia y permite que el resto corra. Esto no ocurre en la implementacion del usuario, si se ejecuta una syscall bloqueante, se bloquea todo el proceso.

Ventajas

Ademas, cada proceso tiene su propio algoritmo de scheduling de threads. Si un thread se bloquea en un join 0 localmente, entonces, como no se requiere bajar al kernel en ese caso se bloquea solo un thread pues la libreria detecta que se esta ejecutando una instruccion bloqueante. Tambien es mucho mas eficiente que la version del kernel debido a que no se debe cambiar de context y tampoco hay flusheos de cache. Esto tambien nos permite decidir nuestro propio algoritmo de scheduling pues estamos en user space.

Desventajas

- Syscall bloqueante
- Page faults, si ocurre bloquea todo, igual que con la anterior
- Inanicion, se soluciona con una analogia timer
- Uso de threads

Implementacion en espacio de kernel

- No es necesario el run-time system ni tabla de threads(en espacio de usuario)
- Un thread se bloquea como es usual y el kernel elige otro thread(u otros procesos)
- Debido al mayor costo, se pueden reutilizar los threads.

Implementacion hibrida

Hay threads en espacio de usuario y threads en espacio de usuario. Por cada thread del kernel, se puede tener varios threads en el espacio de usuario.

Schedulers activators

Uno de los threads(del user space) bloquea todo el proceso, entonces se soluciona con:

- Notificar al kernel(thread manager) para que se entere de la situacion y cree otro thread, y le de el control al run-time system a traves de ese nuevo thread(de kernel).
- Sin embargo, viola la estructura de un sistema de capas con el **upcall** del kernel al user space.

Teoria 5: Scheduling

La responsabilidad del scheduler es simplemente decidir quien es el proximo proceso o thread(indistinguibles en este caso) a correr. La eleccion no es random, generalmente se siguen criterios, conviene empezar con **round robin** y luego hacerlo mas complicado.

Con multiprogramacion se hace referencia a compartiendo la misma memoria, varios procesos corren de forma pseudoparalela y si uno se bloquea se pasa al otro para evitar desperdiciar tiempo(se va de un proceso a otro). Es muy costoso realizar un context switch, se borra la cache, por ejemplo.

Las decisiones de scheduling dependen fuertemente en el comportamiento del proceso, no son agnosticas.

Pregunta parcial: Que me permite determinar de que tipo de proceso estamos hablando? La clave es pensar en la longitud de la rafagas, rafagas cortas es I/O bound y rafagas largas es CPU-bound. Es **importante** NO considerar las rafagas de espera, ya que estas suelen ser similares(Hay que ver las rafagas de CPU). Esto se debe a que lo que hace el proceso esta en el tiempo de cpu, mientras que la longitud del tiempo que esta esperando por I/O depende de otras cosas(no del proceso).

La ejecucion de que proceso va a cambiar el estado en el cual los semaforos estan bloqueados? sem_post de cualquier proceso sobre ese semaforo.

> Con procesadores mas rapidos los procesos se vuelven mas I/O bound. Pues a más ciclos se procesan más instrucciones y la ráfaga de CPU es más corta.

Clasificacion

Non-preemptive

Elige un proceso y corre hasta que se bloquea o hasta que libera el CPU voluntariamente.

Preemptive

Un schedular que evita que sigas teniendo el control del procesador quitandotelo, el scheduler de cualquier sistema. Elige un proceso y corre hasta que se vence un plazo establecido, incluso si esta en estado ready. Es necesaria la interrupcion del timer para darle el control al SO(scheduler).

> TO provent something from happening by taking action first.

La traduccion de apropiativo y no apropiativo no le gusta al profe porque es mas una pripiedad del proceso y no del scheduler. Prefiere la definicion en ingles.

Categorias

Las decisiones de scheduling para diferentes tipos de sistemas operativos pueden variar signitivativamente.

- **Batch**: Non-preemtive o preemptive con quantum largo. Sistemas no interactivos, entonces no es relevante la velocidad de respuesta en interacciones con el usuario.
- **Interactivo**: Preemptive, porque sino podria causar inanicion o, mas general(y comun), para usar un programa habria que esperar a que otro termine y seria muy dificil de usar la computadora(cambiando entre programas como whatsapp y mail).
- **Real time**: A veces no necesita preemptive, uso especifico.

En batch, porque eligiria un quantum mas largo? Porque se evita el costo de muchos context switch, basicamente hay menos overhead de switching entre procesos. Esto se debe a que no necesita que sean tan responsivos los programas.

Objetivos

Puntos a tener presentes para cada tipo de scheduler:

- Fairness
- Policy enforcement
- Balance

Nosotros vamos a tener un scheduler con prioridades, asi que es muy importante que pase el test.

Para los sistemas batch:

- Throughput(realizar la mayor cantidad de trabajos)
- Turnaround time(Tiempo medio que la tarea pasa en el sistemaa, esta presente)
- CPU utilization(Tener el CPU acupado todo el tiempo)

Maximizar throughput minimiza el turnaround time?

Para maximizar el throughput hay que elegir las tareas mas cortas, entonces las tareas largas se quedan colgadas todo el tiempo(casi como si fuera inanicion).

Como en un sistema por lotes tengo la informacion del tiempo total que necesita una tarea para tomar las decisiones, entonces se puede implementar un algoritmo para manejar entre las largas y las cortas.

Para los sistemas iteractivos:

- Responsive time
- Proportionality(cumplir con las expectativas del usuario)

  Si se pone lento Chrome, no es culpa de Chrome, sino que es culpa del SO pues es el encargado de administrar los recursos.

Para los sistemas de tiempo real:

- Meeting deadlines(No perder informacion)
- Predictability(No perder calidad en reproduccion de multimedia)

Algoritmos

Batch: First-Come First-Serve

Un proceso corre hasta que termina, pero si se bloquea va al final de la cola. Es un algoritmo sencillo de programar, pero hay un caso patologico: TEngo un proceso CPU-Bound que corre por 1s y se bloquea en disco y varios proceso I/O-bound que realiza 1000 accesos a disco cada uno.

Durante ese segundo solo se usa el CPU y los otros bloqueados en disco, voy a tener un pedazo de tiempo usando solo CPU y otro con solo I/O. Con esto no logro un balance ideal.

Batch: Shortest job first

El tiempo que demora un parametro en ser completado es conocido en batch. Entonces, se aprovecha eso. Es non-preemptive y optimo si todos los procesos estan disponibles simultaneamente, pero no funciona bien si el tiempo de llegada es diferente(No queda otra mas que esperar a que el primero termine).

Se puede calcular el turnaround time(ver apuntes) y observar que el orden en el que se ejecutan los procesos afecta el turnaround time(Esto asume que llegan al mismo tiempo).

Batch: Shortest remaining time next

Version preemprive de shortest job first. Al que este mas pronto por terminar lo atiende. Al llegar un nuevo trabajo se copmara su tiempo restante(total) con el restante del trabajo actual.

Tengo N tareas en el sistema, me llega una tarea nueva, no deberia compararlo con el tiempo restante de todos los trabajos y elegir al menor?

Basicamente, el que se esta ejecutando ya es el que tiene el menor tiempo restando, por eso la comparacion es con el proceso actual.

Que metrica beneficia este algoritmo?

El troughtput porque siempre saca la tarea mas corta posible. Esto genera un problema en el caso que exista una tarea muy larga y que lleguen siempre tareas mas cortas.

Interactivo: Round-Robin

Es preemptive(el que tenemos que implementar con algun que otro agregado), es la version preemptive del Batch:FIFS.

A cada proceso se le asigna un intervalo de tiempo(quantum) para correr), si al finalizar sigue ready, se le quita el CPU de todos modos. Si se bloquea antes de que venga el quantum, se pasa al siguiente.

Se van corriendo en orden de llegada, pero de a pedazos y se repite ciclicamente(como aplicar FIFO un monton de veces). Es facil de implementar con una lista de procesos ready.

El tamanio del quantum es importante. Por que?

Porque el switcheo no es gratis y porque al ser interactivo tampoco conviene que haya quantums muy grandes(Eso convendria mas para batch). El costo lo podriamos pensar como la relacion entre el tiempo del quantum y el tiempo del context switch. (context switch de 1ms y quantum de 4ms contra un quantum de 100ms).

Entre 20 y 50 ms es razonable.

Interactivo: Priority scheduling

Round-Dobin aume que todos los procesos son igualmente importantes, en este caso se le asignan prioridades a los procesos y aquellos con la maxima prioridad son elegidos. Es preemptive, atiendo siempre al proceso con mayor prioridad.

Para que necesita ser ajustada la prioridad periodicamente?

Para evitar inanicion, pues un proceso con muy poca prioridad podria no ejecutarse. En un sistema interactivo las prioridades cambian dinamicamente a partir de:

- Del usuario(depende de el)
- Del comportamiento del proceso(para I/O-bound puede ser 1/f, donde f es la proporcion del ultimo quantum usado). Esto implica que los CPU-bound tendran menos prioridad.

Que sentido tiene aplicar esta tecnica?

Prioriza los procesos que menos usan el CPU para maximizar el uso de los recursos, es decir, el balance.

Tiene prioridad porque si esta ready lo atiendo para que se bloquee inmediatamente, poniendo el disco a laburar mientras uso el CPU para otra cosa. Logrando un buen balance de carga de todos lo recursos. De alguna manera es inmune a los procesos que quieran usar todo el CPU.

Interactivo: Priority scheduling - Multiple queues

Similar al anterior, solo que se agrupan procesos por clase, donde para procesos de una misma clase se aplica la politica de round robin.

En round robin deja de ser importante quien es el primero, porque se repite constantemente y estar primero o segundo es lo mismo que estar ultimo.

Con cierta frecuecia le le asigna un quantum muy grande a un proceso CPU-bound, y cuando usa todo su quantum se le baja la prioridad. Basicamente, si un proceso usa todo su quantum, es muy probable que sea CPU-bound.

Interactivo: Priority scheduling - Shortest process nect

> No hace falta que se aprendan los nombres de memoria, me importa que lo sepan.

Es la contraparte interactiva de shortest remaining time next, en lugar de basarse en informacion conocida se estima el comportamiento previo. La estimacion se hace basandose en ejecuciones historicas, para intentar se adivinar cuanto va a ejecutar.

Una manera de hacerlo es con una tecnica llamada **envejecimiento o aging**, dandole mas peso a las ejecuciones con menor tiempo.

> **Nota**: En la diapo T0 es un tiempo real, pero le dice estimacion porque se usa el valor real para estimar el siguiente.

Ejemplo parcial: Se registraton los tiempos de las ultimas ejecuciones, con un valor de a de X determinar cuanto va a demorar la proxima ejecucion.

Interactivo: Guaranteed scheduling

Prometamos que si hay n usuarios conectados, cada uno recibia 1/n del tiempo del CPU. Alternativamente, si 1 usuario tiene n procesos, cada proceso recibira 1/n del tiempo del CPU. Luego, registrar el tiempo de uso de cada usuario / proceso en terminos de la proporcion usada para elegir aquel proceso con la menor proporcion.

Trata de garantizar una distribucion igualitaria de tiempo de CPU, dando prioridad siempre a aquel con la menor proporcion.

Interactivo: Lottery Scheduling

El siguiente a correr es una loteria, repartimos tickets a procesos y se sortea el uso del CPU. Para darle mas prioridad a un proceso, simplemente hay que darle mas tickets para que tenga mas probabilidad de ganar.

Para que podria servir esto?

Para algunos contextos es mas sencillo asignar prioridades, para darle el doble de prioridad a un proceso solo hay que darle el doble de tickets.

Procesos que cooperan entre si pueden compartir tickets. Por ejemplo: upongamos que en el campeonato de ChompChamps hay una categoria donde el bot que hicimos compite contra si mismo. Si puede detectar que es el mismo bot(su oponente), entonces se asigna uno como ganador y el ganador recibe los tickets del que se asigno como perdedor.

La intervencion de quien seria necesaria para que se puedan compartir los tickets?

Es necesaria la intervencion del SO mediante syscalls.

Interactivo: Fair-share scheduling

Si empieza a tener presente la nocion de usuario, si un usuario tiene 50 procesos y otro solo 1. Primero se corre un proceso del primero, luego un proceso del segundo, etc.

Si uno tiene procesos ABCD, y el otro solo tiene a E:

A E B E C E D E A B E….

> **Nota**: Correr `cat &` se bloquea y termina con una signal, esto ocurre porque no tiene sentido leer en background.

Para saber si un proceso esta bloqueado o ignorando una signal, el kernel simplemente se fija como esta configurado el proceso que hizo la syscall.

Teorica 6: Memoria

Para cumplir con sus responsabilidades(Teorica 1) el S) realiza:

- Abstraccion
- Interposicion(ponerse entre dos procesos)

La manera mas simple de ambos es que no haya, que sean nulos.

Problemas que pueden ocurrir

Cada proceso accede directo a memoria fisica

1. **Colisiones:**

Primero que todo, van a tener colisiones, es muy dificil tener multiples procesos ejecutando simultaneamente, porque cada uno decidiria que bloque de memoria van a usar(y no necesariamente coordinan, pero hay casos en los que si).

2. **Memoria insuficiente:**

Otro problema, es que puede haber programas que necesiten mas memoria que la que haya disponible.

Ya sea diskettes o Overlay, quien se encarga de particionar los programas?

El programador, es un trabajo muy importante. Hoy en dia, cuando un programa no entra en memoria lo resuelve el sistema operativo.

3. **Programas cargados parcialmente en memoria:**

Por ultimo, es dificil ejecutar programas que no estan completamente cargados en memoria.

4. **Cada proceso accede directamente a la mmisma libreria:**

Es muy dificil que se pongan de acuerdo, entonces conviene que cada uno tenga su propia copia. El linkeo estatico es meter todos los asrchgivos de la libreria en el mismo binario.

Hoy en dia se usa el linkeo dinamico, ocupando menos memoria RAM, **libc**(2-5MB) tiene una sola copia en memoria en todo momento, la cual es compartida por los programas. Haciendo que los programas ocupen menos memoria hasta dentro del disco pues no necesitan tener la libreria en el binario. Es necesario que la libreria este en el sistema si esta linkeada dinamicamente, tambien tienen que tener la misma version para la cual se compilo el programa. Una version puede tener un error que otra version no.

Una ventaja de linkeo estatico, no hay que averiguar donde esta es **muy importante** saber donde esta la libreria dinamica a la hora de ejecutar el programa.

Entonces, la libreria(dinamica) puede no estar o tener otra version en el sistema, y eso es un problema.

5. **Programa que maneja direcciones fisicas obsolutas:**

Es muy dificil que este tipo de programas funcione si no esta cargado exactamente donde esta. Para que funcione es necesario conocer a priori la organizacion de la memoria.

6. **Fragmentacion:**

Es muy dificil de imperdir que la memoria se fragmente, que cresca el heap significa que malloc hace la syscall para aumentar la memoria disponible.

Es muy dificil manejar esto si no existe un mecanimo para permitir que cresca dinamicamente.

7. **Establecer permisos para diferentes areas de la memoria:**

Se relaciona con el problema del principio: Quien protege al SO? Estaria bueno que haya secciones que no se puedan tocar directamente.

Lectura y ejecucion en syscalls, hace falta estructuras en memoria que sean modificables por las syscalls, un proceso debe tener mapeado en su espacio de direcciones la ubicacion de estas estructuras para saber que permisos tienen y donde.

Como se evita que el proceso no pueda modificar dichas estructuras si las syscalls deben tener permisos de escritura?

Hay que ejecutar las syscalls de manera indirecta(ya que si son parte del proceso es un problema), hay que saber diferenciar entre llamas a funcion del programa y llama a funcion de kernel.

Con interrupciones, el handler es codigo de kernel y ahi esta todo bien. Y no se puede cambiar el RIP directamente porque no se puede hacer ya que no hay permisos de acceso a la zona de memoria del kernel. El hardware no lo permite.

> **Nota**: Todos y cada uno de los accesos a memoria son chequeados.

Ahora vamos a cambiar y como abstraccion vamos a tener memoria virtual, mientras que como interposicion, vamos a tener completa(con ayuda del hardware). La idea central de memoria virtual es separar el espacio de direcciones virtuales(EDV) del fisico(EDF).

Espacio de direcciones

Necesario para entender memoria virtual. Estamos poniendo una capa entre medio que realiza el mapeo de memoria.

Como se implementa memoria virtual

El EDV se presenta a cada proceso como un bloque **contiguo** y **exclusivo** de memoria, similar a lo que ocurre en el quantum de CPU.

Mientras que espacio de direcciones fisica no es visible por los procesos, el SO centraliza la administracion del espacio fisico y se encarga de mapear la memoria virtual a la fisica. Esto se puede interpretar como una funcion matematica que ademas **verifica permisos**(lectura, escritura y ejecucion). Cada acceso a memoria es supervisado por la MMU, es lo que verifica que la interposicion sea completa. Cada direccion virtual que sale del procesador se traduce a una direccion fisica chequeando los permisos con la MMU.

> **Nota**: Todos y cada uno de los accesos a memoria deben ser auditados por el kernel.

Como se resuelven los problemas

El SO evita colisiones simplemente no dando el mismo bloque de memoria a dos procesos distintos.

Por otro lado no mapea areas de memoria que no deben ser accedidas.

Una libreria en el EDF puede estar mapeada en multiples EDV.

El SO puede retringir los permisos de cada area de memoria mapeada.

Areas fragmentadas del EDF pueden ser mapeadas desde un area continua en el EDV al igual que areas que crecen dinamicamente.

Si no hay memoria suficiente usa memoria virtual, guardando en disco para hacer espacio. No es un problema tener un programa que use mas memoria de la que esta instalada.

Similarmente, todo el codigo del programa no tiene porque estar completamente cargado en memoria.

En conlusion, hasta ahora nunca tubimos que ocuparnos del pasaje de memoria virtual a fisica de manera que sea transparente para el proceso(similar a lo que hacia overlay).

> Cada procesos tienen sus propios espacios de direcciones(diferentes!), esto es pregunta de examen.

Es decir, cada proceso tiene una unica funcion propia(funcion en el sentido matematico).

Hasta ahora nunca hablamos de paginas, hasta podriamos haber estado hablando de una subdivision a nivel de bytes(aunque no sea conveniente). Esto se debe a que memoria virtual y paginacion son cosas totalmente distintas(que sualen ser mostradas en la misma teoria, pero podes tener cada uno por su cuenta).

De manera que memoria virtual soluciona colisiones, falta de memoria, librerias compartidas. Esas son problemas que soluciona, no necesariamente ventajas.

Paginacion

Ahora comenzamos a hablar de granularidad con permisos y todo lo demas. Entonces, es agrupar bytes contiguos en bloques de igual tamano.

Todos los permisos van a ser a nivel de pagina, no podriamos hacer que parte de pagina si y parte de pagina no.

La pagina virtual esta en memoria virtual y el marco de pagina esta en memoria fisica(VP y PF respectivamente).

Tabla de paginas

Es basicamente la definicion de la funcion de memoria por partes, donde cada parte es una pagina. Posee una entrada por cada VP, traduce de a paginas enteras.

La DV(seria dentro de la MMU) se divide en dos partes:

- **Bits mas significativos**: Numero de VP(indice en la tabla).
- **Bits menos significativos**: Offset dentro de la pagina.

En la entrada de la tabla hay varias flags que indican:

- PF number
- Present/absent(para cargar dinamicamente el codigo de un programa, indica si se guardo alguna vez en disco o nunca se cargo)
- Protection
- Modified
- Referenced(indica si se referencio en el pasado cercano, se usa para saber cual bajar a disco).

Los bits se verifican en el momento que se produce(en especial el de proteccion).

> **Nota**: Hay **una** tabla de paginas por proceso.

Page fault

Ocurrio alguna violacion de acceso, generalmente salta debido a estos bits:

- Present/absent
- Protection

La MMU lanza una excepcion antes de que se acceda a la memoria y le pasa el control al SO:

- Si el PF no esta presente, se trae del disco(hard) o se mapea si ya esta en memoria(soft).
- Si se violan los permisos, se mata el proceso. A menos que sea un caso de COW.
- Si se accede a una VP que no esta mapeada directamente, se mata el proceso. A menos que hablemos de **asignacion lazy**(en este caso el kernel no lo cargo realmente y lo tiene que hacer en ese momento, es decir, no lo hace hasta que sea estrictamente necesario).

  Not in memory **USUALMENTE** significa que no esta en memoria. Caso contrario, significaria que esta en otro lugar(el kernel lo puede guardar como quiera, mismo con el estado de un proceso en un context switch, no necesariamente tiene que ir a disco o a memoria).

No todo es color de rosa

Tenemos el dilema de como manejar la memoria libre, hay que usar un algoritmo para determinar donde colocar las cosas, que esta libre y que esta ocupado. Esto ya lo tubimos que solucionar en el TP02, hay dos enfoques:

- **Bitmap**: se divide la memoria en bloques y cada bit representa su estado.
- **Free list**: cada proceso y espacio libre tiene un nodo con inicio, longitud y el puntero al siguiente.

Pero, como se busca el bloque libre? Hay varias opciones:

- First fit(Comienza desde el principio)
- Next fit(Comienza donde quedamos)
- Best fit(Bloque que mejor se adapta)
- Worst fit(Buscar bloque mas grande)
- Quick fit(4 free lists de bloques de 4KB, 8KB, 12KB y 16KB. Elije el primer bloque de la lista correspondiente)

Porque worst first tiene sentido? Porque best fit generalmente deja un pedazo muy chico de memoria sin usar(generalmente no se encuentra exactamiente el bloque requerido). Eso es muy malo y causa mucha fragmentacion, worst fit no hace eso y deja bloques libres razonables. De manera de worst fit es mejor que best fit.

El problema de quick fit es que va a haber block sizes mas demandados que otros, entonces se van a acabar mas rapido mientras los otros sobran.

Por otro lado, la traduccion tiene un costo(por mas que es a nivel de hardware). Cada acceso a memoria debe traducirse.

Tercer punto, si la EDV es grande, la tabla de paginas sera grande.

Finalmente, con que criterio se determina que pagina se baja a disco.

Analisis de costos

Hay que tener dos cosas en cuenta:

- La traduccion de DV a DF debe ser rapida.
- Si el EDV es grande, la tabla de paginas sera grande.

Habria que hacer dos lecturas a memoria por cada lectura, es decir, hay una lectura extra la cual involucra buscar la traduccion de la direccion virtual a la direccion fisica en la tabla de paginas.

Con DV de 32 bits tendriamos una tabla de paginas de 4MB, esto es mucho. Entonces hay que buscar un punto medio, que no este totalmente en memoria y que no este totalmente en un registro.

TLB

Esto se soluciona con una **Translation Lookaside Buffer**, pero primero, que ocurre?

- Los procesos hacen muchas referencias a pocas paginas.
- Los procesos hacen pocas referencias a muchas paginas.

Debido a que el codigo se encuentra contiguo, lo mismo ocurre con el stack, termina siendo verdadera la primera opcion.

Es mucho mas chica, ya que un proceso accede a un conjunto chico de paginas, asi que solo tiene 256 entradas. Es muy parecido a la tabla de paginas. El primer acceso a una pagina va a terminar funcionando como cuando teniamos antes, con dos accesos a memoria. Sin embargo, el segundo acceso ya va a ser con la TLB, la cual tendra una entrada generada por la primera vez. Si se queda sin espacio, habra que descartar alguna entrada.

Cuando viene un pedido, hay un hardware especial que permite comparar todas las entradas de la TLB de manera simultanea. Chequea permisos y devuelve la direccion fisica.

> Es exactamente el trabajo que hace la tabla de paginas.

Una entrada de la TLB desalojada no hay que pisarla simplemente, sino que hay que actualizar la tabla de paginas pues podria haber cambiado el bit **modified** o el **page frame**(notar igual que el bit de presencia no esta en la TLB).

(Investigar mejor, ver clase alrededor de 19:15, porque se modifica el page frame).

De todas maneras, todavia no solucionamos el problema de una EDV muy grande, hay dos enfoques:

- Tabla de paginas multinivel
- Tabla de paginas invertidas

Tabla de paginas multinivel

Evita tener toda la tabla en memoria, solo tenemos lo necesario. Basicamente es lo mismo que el directorio de tablas de paginas visto en arqui.

En el nivel principal(PT1) tengo 2^10 entradas, luego en el PT2 otras 2^10 y finalmente el offset de 2^12 posibles valores.

Cuanto ocupa cada tablita? 2^10 * 4B, oh casualidad, entra exactamente en una pagina. Esto no es casualidad, sino que fue pensado deliberadamente. Entonces, no tengo todas las entradas, puede empezar

con cuatro tablas y listo. En caso de necesitar una pagina que no este, vamos a tenes que reservar una nueva pagina para la nueva tabla, pero sigue siendo mucho menos espacio.

Esto fue usado por Intel, lo que conocemos como page directory - page table, tambien conocido como esquema 2^10 2^10 2^12. Diez anos despues se extienden las entradas a 64 bits con el Pentium Pro, entonces se uso un esquema 2^9 2^9 2^2 2^12, basicamente para que entre en una pagina.

Sin embargo, ahora que se incorporo el soporte completo a 64 bits se agrega un cuarto nivel con un esquema 2^9 2^9 2^9 2^9 2^12 = 2^45. Page Map Level 4.

A partir de 2017, se incorpora un quinto nivel de paginacion en algunos procesadores llamado PLM5 (Page Map Level 5). Con esquema 2^9 2^9 2^9 2^9 2^9 2^12 = 2^52.

Tabla de paginas invertidas

Hasta ahora las tablas que vimos tenian una entrada por cada pagina, pero ahora tenemos una entrada por cada PF en lugar de cada VP. Ahora hay que buscar toda la pagina hasta encontrar la VP, para solucionar esta busqueda se usa una tabla hasheada.

La TLB sigue sirviendo, una vez tenemos la informacion se puede guardar en la TLB. Sin embargo, una TLB miss requerira recorrer toda la tabla para encontrar la VP. Si no se encuentra una VP, entonces es similar a un Page Fault y se debera buscar un lugar libre y actualizar la tabla.

Algoritmo de reemplazo de paginas

Con memoria virtual(no paginacion) podemos darnos el lujo de no tener todo el proceso en memoria para que funcione y cuando hace falto un PF que no esta presente se trae del disco a memoria. Ahora, que PF vamos a desalojar?

Optimo

Es facil de describir, pero imposible de implementar(no se puede predecir el futuro). Si se pudiera clasificar las paginas de acuerdo a cuando van a volver a ser referenciadas en el futuro, entonces se va a desalojar a que que tardara mas en ser usado. Se quiere postergar traer una pagina del disco tanto como sea posible.

Entonces, para que se menciona si es imposible? Porque se puede usar para saber que tan cerca se esta del optimo, al ser el optimo tenemos una cota superior de lo mejor que podria ser un algoritmo.

Pero, como se puede medir que tan optimo es un algoritmo? Podemos evaluar el optimo haciendo un poco de trampa: Hacer una ejecucion en condiciones controlares y medir cuando se accede a cada pagina. Luego repetir la ejecucion, pero usando la informacion obtenida para saber que pagina hay que desalojar. Una manera de evaluar esto seria midiendo el overhead o cantidad de accesos a discos.

Not recently used(NRU)

Usa los bits R(referenciado) y M(modificado) para tomar las decisiones de reemplazo. El bit R se resetea periodicamente, pues sino no sirviria mucho y resetearlo permite saber si se referencio hace poco. Pero no se resetea M ya que se usa para otra cosa, si no se modifico entonces no hace falta actualizar su version en disco y se evita el costo.

En funcion a los dos bits tenemos cuatro clases diferentes:

- **Clase 0**: ~R & ~M
- **Clase 1**: ~R & M
- **Clase 2**: R & ~M
- **Clase 3**: R & M

De manera que se elije dependiendo de la clase, prefiriendo la clase 0 antes que la 1, etc.

First-in first-out(FIFO)

Simplemente se registra el tiempo en el cual cada pagina fue cargada en memoria y de desaloja la que mas tiempo estuvo en memoria.

Pero bueno, que sea la primera no implica que sea la menos usada. Cuando se quita la mas antigua, puede que se vuelva a usar y pasa a ser la mas nueva.

Second Change(SC)

Modifica el FIFO y le agrega el bit R. Se sigue usando reseteando en R periodicamente y si la ultima pagina tiene el bit en 1, se la modifica y se la manda al comienzo de la fila sin tener que bajarla a disco. Si todas fueron referenciadas, vuelve a ser FIFO comun.

Clock(C)

....

Least recently used(LRU)

Las paginas muy usadas recientemente suelen seguir siendo muy usadas, mientras que las escasamente usadas recientemente suelen seguir sin ser muy usadas.

Desaloja la pagina que no a sido usada el mayor tiempo, no es mas que una estimacion de lo que ocurrira en el futuro. Tiene dos enfoques:

- Tener una lista de paginas ordenada por tiempo de acceso. Desalojar a la mas antigua.
- Tener un registro especial que cuente instrucciones y equipar a cada entrada de la tabla con espacio para este registro. Cuando se referencia una pagina se actualiza su contador. Desalojar a la que tenga el contador mas chico.

Not frequently used(NFU)

Se requiere de un contador por software para cada pagina y periodicamente se suma el bit R de cada pagina a su contador. Entonces, se desaloja la pagina con el contador mas chico. Durante toda la vida del proceso se mantiene el contador, asi que nunca se va a desalojar la pagina incluso si el proceso dejo de usarla. Hay que esperar hasta que otras paginas le ganen.